

Integrating Alpha-Based Optimal Trading and Advanced Methods for SPY Options

Integrating the Passerini–Vázquez HJB Framework and Modern Quantitative Methods into a High-Frequency SPY Options Trading System

Executive Summary

This report was originally presented as an extension to former research in 2018 by the author. It presents a comprehensive, technically rigorous roadmap for integrating the methods and systems described in Passerini and Vázquez’s “Optimal Trading with Alpha Predictors” (arXiv:1501.03756) into a modern SPY options trading system. The Passerini–Vázquez framework leverages the Hamilton–Jacobi–Bellman (HJB) equation to derive optimal trading strategies under linear costs and temporary impact, introducing the concepts of no-trade (NT) zones and market-making zones for market and limit orders, respectively. This report details the mathematical underpinnings of the framework, provides explicit derivations and implementation recipes, and then adapts these methods to the unique microstructure and risk profile of SPY options trading.

To maximize net profitability, execution efficiency, and alpha capture, the report also synthesizes recent advances from quantitative trading literature—spanning deep reinforcement learning, deep hedging, advanced market-making, and machine learning-based fill probability estimation. Each method is compared in terms of theoretical foundation, empirical performance, and practical feasibility for real-time, high-frequency SPY options trading. The report concludes with a step-by-step integration strategy, Python code snippets for key components, and a set of recommendations for constructing a machine–machine interface that pushes the boundaries of current quantitative trading technology.

1. Introduction and Problem Context

The SPDR S&P 500 ETF Trust (SPY) is the most liquid and actively traded ETF and options product in the world, with daily options volume exceeding 8 million contracts and accounting for a significant fraction of total US options activity. This liquidity, combined with tight bid–ask spreads and a deep options chain, makes SPY options an ideal testbed for advanced algorithmic trading strategies. However, the high-frequency, low-latency environment of SPY options also presents unique challenges: execution costs, slippage, partial fills, and the need for robust risk and inventory management.

Passerini and Vázquez (2015) propose a general HJB-based framework for optimal trading with alpha predictors, linear costs, and temporary impact, supporting both market and limit orders.

Their approach introduces the concept of NT and market-making zones, and provides explicit, implementable recipes for optimal trading under realistic constraints. While their original context is single-asset trading, the framework is highly adaptable to options trading, particularly when combined with modern advances in machine learning, reinforcement learning, and market microstructure modeling.

This report aims to:

- Explain the Passerini–Vázquez framework in detail, including mathematical derivations and implementation recipes.
- Adapt the framework to the specific requirements of SPY options trading, including alpha modeling, execution cost modeling, and order type selection.
- Survey and compare recent superior methods from quantitative trading literature, focusing on reinforcement learning, deep hedging, and advanced market-making.
- Provide a step-by-step integration strategy, including Python code and scientific explanations, for building a cutting-edge SPY options trading system.

2. The Passerini–Vázquez HJB Framework: Core Concepts and Mathematical Derivation

2.1. Problem Setup and Notation

The Passerini–Vázquez framework considers trading a single asset (here, SPY or a specific SPY option) over a finite horizon, typically from market open to close. The price process P_t follows:

$$dP_t = \mu_t dt + \sigma dW_t$$

where μ_t is the drift (alpha), decomposed into a constant daily component μ and a fast, mean-reverting intraday component x_t :

$$\begin{aligned}\mu_t &= \mu + x_t \\ dx_t &= -\kappa x_t dt + \eta dZ_t\end{aligned}$$

The trading position q_t evolves as:

$$dq_t = u_t dt$$

where u_t is the trading rate (shares or contracts per unit time).

The objective is to maximize expected P&L, penalized by linear trading costs (half-spread C), temporary impact (quadratic cost Ku_t^2), and risk (variance penalty λ):

$$V(t, x, q) = \min_u \mathbb{E} \left[\int_t^T (C |u_s| + Ku_s^2) ds - \int_t^{2T} (\mu + x_s) q_s ds + \frac{\lambda}{2} \int_t^{2T} q_s^2 ds \right]$$

2.2. HJB Equation and Value Function

Applying the dynamic programming principle, the value function $V(t, x, q)$ satisfies the HJB equation:

$$\mathcal{D}_{t,x}V + \frac{\lambda}{2}(q - q^*)^2 + \min_u \left[C |u| + Ku^2 + u \frac{\partial V}{\partial q} - gu \right] = 0$$

where:

- $q^* = \mu/\lambda$ is the daily Markowitz portfolio (target position).
- $g(t, x) = \mathbb{E} \left[\int_t^{2T} x_s ds \mid x_t = x \right]$ is the integrated gain from the fast signal.

Derivation of the No-Trade Zone

Ignoring the quadratic impact term ($K \rightarrow 0$), the value function simplifies to:

$$V(t, x, q) \approx \frac{1}{2}(2T - t)(q - q^*)^2$$

$$\frac{\partial V}{\partial q} = (2T - t)(q - q^*)$$

The NT zone boundaries are:

$$b_{\pm}(t, x) = q^* \pm \frac{g(t, x) - C}{(2T - t)}$$

If the current position q is within $[b_-, b_+]$, **do not trade**; otherwise, trade instantly to the nearest boundary.

Trading Rate with Temporary Impact

When $K > 0$, the optimal trading rate is:

$$u^* = \frac{1}{2K} \left[(g - C - \frac{\partial V}{\partial q})^+ - (\frac{\partial V}{\partial q} - g - C)^+ \right]$$

This formula ensures that trading is aggressive when far from the NT zone and slows as the boundary is approached.

2.3. Extension to Limit Orders and Market-Making Zones

When both market and limit orders are allowed, the position evolves as:

$$dq_t = (m_t^+ - m_t^- + 1_t^+ l_t^+ - 1_t^- l_t^-) dt$$

where:

- $m_t^\pm$: market order sizes (buy/sell)
- $l_t^\pm$: limit order sizes (buy/sell)
- $1_t^\pm$: indicator of limit order fill (depends on short-term price movement)

The HJB equation becomes:

$$\mathcal{D}_{t,x,y}V + \frac{\lambda}{2}(q - q^*)^2 + \min_{m,l} [m^+(C + \frac{\partial V}{\partial q} - g) + P^+l^+(C + \frac{\partial V}{\partial q} - g) + m^-(C - \frac{\partial V}{\partial q} + g) + P^-l^-(C - \frac{\partial V}{\partial q} + g)] = 0$$

where $P^\pm$ are fill probabilities for limit orders.

Five Trading Regions:

1. **Buy Market:** $g > C(1 + P^+)/P^+ + \frac{\partial V}{\partial q}$
2. **Buy Limit:** $C + \frac{\partial V}{\partial q} < g < C(1 + P^+)/P^+ + \frac{\partial V}{\partial q}$
3. **Market-Making:** $|g - \frac{\partial V}{\partial q}| < C$
4. **Sell Limit:** $C(1 + P^-)/P^- - \frac{\partial V}{\partial q} < g < C + \frac{\partial V}{\partial q}$
5. **Sell Market:** $g < C(1 + P^-)/P^- - \frac{\partial V}{\partial q}$

Zone Boundaries:

$$b_\pm(t, x) = q^* \pm \frac{g(t, x) - C}{(2T - t)}$$

$$\tilde{b}_\pm(t, x, P) = q^* \pm \frac{g(t, x) - C}{(2T - t)(1 \pm P)}$$

2.4. Implementation Recipes and Pseudocode

Algorithmic Steps:

1. **At each decision time t :**
 - Compute current fast signal x_t and integrated gain $g(t, x_t)$.
 - Compute NT/market-making zone boundaries $b_\pm(t, x_t)$.
 - If q_t outside $[b_-, b_+]$, trade (market or limit) to the nearest boundary.
 - For limit orders, estimate fill probabilities $P^\pm$ using short-term predictors.
 - Adjust order type and size according to the five-region logic above.

Python Pseudocode:

```
1. def compute_gain(t, x, kappa, T):
2.     # Ornstein-Uhlenbeck gain
3.     return x * (2*T - t) + (1 - np.exp(-kappa * (2*T - t))) / kappa
4.
5. def nt_zone_boundaries(q_star, g, C, T, t):
6.     width = (g - C) / (2*T - t)
7.     return q_star - width, q_star + width
8.
9. def trading_decision(q, b_minus, b_plus):
10.    if q < b_minus:
11.        return 'buy', b_minus - q
12.    elif q > b_plus:
13.        return 'sell', q - b_plus
14.    else:
15.        return 'hold', 0
16.
17. # Example usage
18. q_star = mu / lam
19. g = compute_gain(t, x, kappa, T)
20. b_minus, b_plus = nt_zone_boundaries(q_star, g, C, T, t)
21. action, size = trading_decision(q, b_minus, b_plus)
22.
```

3. Adapting the Framework to SPY Options Trading

3.1. Modeling Alpha Predictors for SPY Options

SPY options present unique challenges and opportunities for alpha modeling:

- **Daily Alpha:** Derived from macroeconomic indicators, index momentum, volatility regime shifts, and cross-asset flows.
- **Intraday/HF Alpha:** Based on order flow imbalance, implied volatility skew, gamma exposure, and microstructure signals (e.g., bid-ask pressure, volume spikes).

Recommended Approach:

- **Daily Alpha (μ):** Use a combination of statistical factor models and machine learning (e.g., LightGBM, LSTM) trained on macro and options-specific features.
- **Intraday Alpha (x_t):** Model as an Ornstein–Uhlenbeck process, calibrated to high-frequency SPY options data (e.g., 1-min or 5-min intervals), incorporating order book features and implied volatility changes.

Python Example:

```
1. # Fit OU process to intraday alpha
2. from statsmodels.tsa.ar_model import AutoReg
3. def fit_ou_process(signal):
4.     # signal: high-frequency alpha series
5.     model = AutoReg(signal, lags=1).fit()
6.     kappa = -np.log(model.params[1])
7.     sigma = np.std(model.resid)
8.     return kappa, sigma
9.
```

3.2. Execution Costs and Impact Modeling for Options

- **Linear Costs (C):** For SPY options, the half-spread is typically 1–2 cents for liquid strikes, but can widen during volatility spikes or for OTM/illiquid contracts.
- **Temporary Impact (K):** While options are less sensitive to market impact than equities, aggressive orders in thin strikes can move the market. Estimate K empirically via regression of execution price vs. order size, or use conservative estimates for risk control.

3.3. Limit vs. Market Order Strategies

- **Market Orders:** Guarantee execution but pay the spread and risk slippage.
- **Limit Orders:** Capture the spread but risk non-execution or adverse selection.

Fill Probability Estimation:

- Use machine learning (e.g., RNNs, survival analysis) to estimate fill probabilities based on order book state, recent fills, and microstructure features.

Python Example:

```
1. # Estimate fill probability using a trained ML model
2. def estimate_fill_prob(features, model):
3.     return model.predict_proba(features)
4.
```

3.4. High-Frequency and Real-Time Constraints

- **Latency:** In SPY options, execution latency of 100–500 ms is acceptable for most strategies, but lower is better for scalping and market-making.
- **Data Infrastructure:** Use direct feeds (OPRA, exchange APIs), colocated servers, and robust order management systems for minimal latency and high reliability.

4. Comparative Analysis: Recent and Superior Methods from Quantitative Trading Literature

4.1. Deep Reinforcement Learning (DRL) for Optimal Execution

Key References: Hafsi & Vittori (2024), Lalor & Swishchuk (2025)

- **Approach:** Model the trading problem as a Markov Decision Process (MDP), with state variables including inventory, time, order book features, and Greeks. Use DRL (e.g., DQN, PPO, SAC) to learn optimal policies from simulated or historical data.
- **Strengths:** Handles high-dimensional, nonlinear environments; adapts to changing market regimes; can incorporate complex reward functions (e.g., P&L, risk, fill quality).

- **Limitations:** Requires large amounts of data and careful simulation; interpretability can be challenging; real-world deployment needs robust risk controls.

Python Example (using Stable Baselines3):

```
1. from stable_baselines3 import PPO
2. # Define environment with state: (inventory, time, order book, Greeks)
3. env = SPYOptionsEnv()
4. model = PPO('MlpPolicy', env, verbose=1)
5. model.learn(total_timesteps=1_000_000)
6.
```

4.2. Deep Hedging and Model-Free Control

Key References: Buehler et al. (2019), Aubert et al. (2025)

- **Approach:** Use neural networks to directly parameterize the hedging or quoting policy, optimizing via Monte Carlo simulation and automatic differentiation. Can handle path-dependent payoffs, transaction costs, and market impact.
- **Strengths:** Flexible, model-free, and can incorporate realistic market impact and fill dynamics.
- **Limitations:** Computationally intensive; requires careful calibration and validation.

4.3. Advanced Market-Making and Inventory Risk Control

Key References: Avellaneda & Stoikov (2008), Gueant et al. (2020)

- **Approach:** Use HJB equations or RL to optimize quoting strategies, balancing spread capture, inventory risk, and adverse selection. For options, extend to multi-dimensional risk (delta, gamma, vega).
- **Strengths:** Proven in both academic and industrial settings; can be extended to options with appropriate risk metrics.
- **Limitations:** Complexity increases rapidly with number of risk factors; requires robust estimation of order flow and fill probabilities.

4.4. Machine Learning for Alpha and Fill Probability Prediction

Key References: Maglaras et al. (2021)

- **Approach:** Use RNNs or other ML models to predict short-term price moves, fill probabilities, and optimal order placement.
- **Strengths:** Improves execution quality and alpha capture; can be integrated with HJB or RL frameworks.
- **Limitations:** Requires high-quality labeled data; risk of overfitting.

5. Integration Strategy: Step-by-Step Plan for System Adaptation

5.1. System Architecture Overview

- **Alpha Engine:** Generates daily and intraday alpha signals using statistical and ML models.
- **Execution Engine:** Implements HJB-based NT/market-making zone logic, with dynamic selection between market and limit orders.
- **Order Management System (OMS):** Handles order placement, modification, and cancellation, integrating fill probability models.
- **Risk and Inventory Manager:** Monitors Greeks, inventory, and risk limits in real time.
- **Backtesting and Simulation Module:** Supports Monte Carlo and historical replay for strategy validation.
- **Infrastructure:** Low-latency data feeds, colocated servers, robust monitoring.

5.2. Step-by-Step Integration Plan

Step 1: Alpha Modeling and Calibration

- Collect and preprocess SPY options data (prices, volumes, Greeks, order book).
- Fit daily and intraday alpha models (e.g., ARIMA, SARIMAX, ML regressors).
- Calibrate OU process parameters for intraday alpha.

Step 2: HJB Zone Computation

- Implement functions to compute NT and market-making zone boundaries using the formulas from Section 2.
- Integrate fill probability estimation for limit orders.

Step 3: Execution Logic

- At each decision time:
 - Update alpha signals and current position.
 - Compute zone boundaries.
 - Decide on order type (market/limit) and size.
 - Place orders via OMS, using limit order price optimization if desired.

Step 4: Risk and Greeks Management

- Continuously monitor delta, gamma, vega, and other Greeks.
- Adjust positions to maintain risk within predefined limits.
- Integrate with HJB logic to penalize excessive risk.

Step 5: Backtesting and Simulation

- Implement Monte Carlo simulation of price and fill processes.
- Validate strategy performance, including P&L, Sharpe, drawdown, and execution quality.

Step 6: Real-Time Deployment and Monitoring

- Deploy on low-latency infrastructure.
- Monitor execution quality, fill rates, and risk metrics in real time.
- Implement automated alerts and fail-safes.

6. Python Implementation: Scripts and Numerical Solvers

6.1. HJB Zone Computation

```
python
import numpy as np

def ou_gain(t, x, kappa, T):
    return x * (2*T - t) + (1 - np.exp(-kappa * (2*T - t))) / kappa

def nt_zone(q_star, g, C, T, t):
    width = (g - C) / (2*T - t)
    return q_star - width, q_star + width

def trading_decision(q, b_minus, b_plus):
    if q < b_minus:
        return 'buy', b_minus - q
    elif q > b_plus:
        return 'sell', q - b_plus
    else:
        return 'hold', 0
```

6.2. Fill Probability Estimation (ML-based)

```
python
from sklearn.ensemble import RandomForestClassifier

# features: order book state, recent fills, etc.
def estimate_fill_prob(features, model):
    return model.predict_proba(features)[:, 1]
```

6.3. RL-Based Execution Policy (Stable Baselines3 Example)

```
1. from stable_baselines3 import PPO
2.
3. class SPYOptionsEnv(gym.Env):
4.     # Define state: (inventory, time, Greeks, order book)
5.     # Define action: (order type, size, price)
6.     # Define reward: P&L, risk penalty, fill quality
7.     pass
```

```
8.  
9. env = SPYOptionsEnv()  
10. model = PPO('MlpPolicy', env, verbose=1)  
11. model.learn(total_timesteps=1_000_000)  
12.
```

6.4. Monte Carlo Simulation for Backtesting

```
1. def simulate_price_path(mu, sigma, T, dt):  
2.     N = int(T / dt)  
3.     W = np.random.randn(N)  
4.     P = np.zeros(N)  
5.     P[0] = 100 # initial price  
6.     for i in range(1, N):  
7.         P[i] = P[i-1] + mu * dt + sigma * np.sqrt(dt) * W[i-1]  
8.     return P  
9.
```

7. Scientific Explanations and Advanced Theoretical Insights

7.1. HJB as the Foundation of Optimal Control

The HJB equation provides a necessary and sufficient condition for optimality in stochastic control problems. In the context of trading, it encodes the trade-off between immediate execution costs, future alpha, and risk. The NT and market-making zones arise naturally as regions where the marginal benefit of trading is less than the marginal cost, leading to inaction—a phenomenon rigorously justified by viscosity solution theory.

7.2. Machine–Machine Interface: Toward Autonomous Alpha Capture

By integrating HJB-based logic with real-time ML predictors and RL-based execution policies, the system becomes a true machine–machine interface: alpha signals, risk metrics, and execution feedback are continuously processed and acted upon without human intervention. The system can adapt to changing market regimes, learn from execution outcomes, and optimize both profitability and risk in a closed feedback loop.

7.3. Theoretical Extensions

- **Multi-Asset and Multi-Greek Control:** Extend the HJB framework to vector-valued positions and risk metrics, enabling simultaneous control of delta, gamma, vega, etc.
- **Nonlinear Impact and Market Microstructure:** Incorporate nonlinear impact functions and detailed order book dynamics for more realistic modeling.
- **Online Learning and Adaptive Control:** Use online learning algorithms to adapt model parameters and execution policies in real time.

8. Feasibility, Limitations, and Regulatory Considerations

- **Feasibility:** All components described are implementable with current technology and open-source libraries (e.g., NumPy, scikit-learn, Stable Baselines3, JAX, hjb-solver).
- **Limitations:** Accurate fill probability estimation and impact modeling require high-quality, high-frequency data. RL and deep hedging methods need careful validation to avoid overfitting and catastrophic risk.
- **Regulatory:** Ensure compliance with SEC and exchange rules on options trading, risk limits, and fair market practices. Implement robust monitoring and kill-switches.

9. Final Recommendations

- **Adopt the Passerini–Vázquez HJB framework as the core logic for execution, using NT and market-making zones to minimize costs and maximize alpha capture.**
- **Integrate ML-based alpha predictors and fill probability models for SPY options, calibrated to both daily and intraday data.**
- **Leverage reinforcement learning and deep hedging for adaptive, model-free execution in complex or nonstationary environments.**
- **Implement robust risk and Greeks management, with real-time monitoring and automated controls.**
- **Continuously backtest and simulate strategies using Monte Carlo and historical replay, validating performance and risk under diverse scenarios.**
- **Deploy on low-latency, high-reliability infrastructure, with automated monitoring and fail-safes.**
- **Iterate and refine the system using online learning and feedback from live trading performance.**

By synthesizing the rigorous HJB-based approach of Passerini and Vázquez with state-of-the-art machine learning and reinforcement learning methods, and by adapting these to the unique microstructure of SPY options, it is possible to construct a trading system that not only matches but exceeds the capabilities of current industry standards. Such a system, operating as a true machine–machine interface, stands to set a new benchmark for execution efficiency, alpha capture, and risk-adjusted profitability in high-frequency options trading.